

Challenging a 0.6B LLM: Are You Smarter Than a Compact AI?

Jingxuan Sun | 390244 | jingxuan.sun@epfl.ch

Igor Pavlovic | 386589 | igor.pavlovic@epfl.ch

Rongxiao Qu | 389141 | rongxiao.qu@epfl.ch

Andres Eduardo Nowak de Anda | 387415 | andres.nowakdeanda@epfl.ch

Monster Hunter

Abstract

Large Language Models (LLMs) have demonstrated impressive reasoning capabilities but often require task-specific fine-tuning to excel at specialized domains such as STEM. In this project, we aim to transform a pre-trained LLM (Qwen3-0.6B-Base) into an AI tutor capable of reliably solving advanced STEM questions. We explore various post-training techniques, including supervised finetuning (such as instruction tuning) (MCQA), Direct Preference Optimization (DPO), quantization, and retrieval-augmented generation (RAG) to enhance reasoning ability. Experimental results show 5% accuracy improvement across MCQA benchmarks and 18% on preference benchmarks.

1 Introduction

Large Language Models (LLMs) often struggle in domains requiring structured reasoning, such as STEM. While models like GPT-4 show strong performance, their scale and proprietary nature prompt a key question: can smaller, open-weight models be fine-tuned for advanced reasoning tasks?

This project explores how Qwen3-0.6B-Base, a relatively small-scale model, can be adapted into an effective AI tutor for STEM multiple-choice question answering (MCQA). Compact models face several challenges: limited context capacity, format misalignment, and a tendency toward catastrophic forgetting during fine-tuning, especially problematic in tasks demanding multi-step reasoning and factual precision.

To address these challenges, we employ a multi-stage pipeline incorporating instruction fine-tuning, supervised MCQA fine-tuning, DPO, quantization, and RAG. We test various loss formulations including answer-letter cross-entropy and sequence-to-sequence generation, add RAG to mitigate context limitations, and apply DPO to align outputs with human preferences.

2 Approach

Our initial approach focused on instruction tuning to enhance reasoning capabilities. Following Tulu 3 methodology (Lambert et al., 2024a) with adjustments for our smaller model size (Appendix 13, 14). Inputs followed the format shown in A.4. We performed two experiments, one with loss computed over the entire output sequence, and the other computed on completion segment only. Evaluation during training was conducted on a held-out 5% split of the training data and using the MMLU benchmark. However, instruction tuning yielded only marginal improvements in MCQA performance, likely because of the size of the model and due to a mismatch between the fine-tuning dataset and the multiple-choice answer format required at evaluation time. (Appendix 11)¹

Given these limitations, we turned to supervised fine-tuning strategies that explicitly aligned the model with the MCQA task. Specifically, we experimented with: fine-tuning on concatenated question-answer pairs (seq2seq), cross-entropy training 1 over all answer options and doing reinforcement learning with verifiable rewards on top of the seq2seq model using RLVR with GRPO (Shao et al., 2024). The reward function assigned +2 for correct letter and explanation (e.g., A. [text]), +1 for the letter only, and -1 for missing answers.

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log \left(\frac{\exp(s_{i,y_i})}{\sum_{j=1}^{O_i} \exp(s_{i,j})} \right) \quad (1)$$

where:

B : Batch size, O_i : Options for example i

$s_{i,j}$: Logit for option j , y_i : Correct option index

These methods were applied to both the base model and the instruction-tuned variants. The best results were obtained by fine-tuning the base model

¹MCQA Project Repository

directly using the cross-entropy loss over all options and formatting examples with random templates A.1.3.

During evaluation, we observed a systematic bias in the base model: it favored more options A–D when presented with questions containing more than four choices. (Appendix B.3) To mitigate this, we generated a synthetic training dataset by augmenting the MMLU-STEM training data with additional distractor options created using ChatGPT. Incorporating this augmented training dataset improved to MCQA (19.2%) and RAG (21%) performance on MMLU-PRO. However, since the evaluation set always has 4 answer options this data was not used for training the final RAG model.

For the retrieval-augmented generation (RAG) component, we constructed a retrieval corpus comprising 100,000 randomly sampled document chunks drawn from Wikipedia (milkshake721, 2024), open-access STEM textbooks (Lab, 2023), and StackExchange forums (Embeddings, 2023). Document embeddings were obtained using the BAAI/bge-small-en-v1.5 encoder (BAAI, 2024).

The RAG training pipeline closely followed the MCQA fine-tuning approach, with the key difference being the inclusion of retrieved documents in the model input (Appendix A.4). We used the MCQA fine-tuned model from M2 as the base and trained it further to effectively leverage the retrieved context during answer generation. The hyperparameters used for fine-tuning the RAG model are listed in Appendix 23. In addition to fine-tuning the language model, we experimented with retriever fine-tuning using the RePlug LSR framework (Shi et al., 2023). However, retriever fine-tuning did not yield noticeable improvements, likely due to the relatively small size of the retrieval corpus, which already ensured high coverage of existing relevant documents in the retrieved context.²

For Direct Preference Optimization (DPO), we employed a length-normalized loss (Meng et al., 2024) and investigated various configurations, including different reference models, varying β parameters to control the deviation of the DPO model, diverse chat templates, and distinct dataset mixes. We used 1% of the data as a validation dataset during training. The model that exhibited the best overall performance on the evaluation set was se-

lected as our final DPO model.

3 Experiments

Data: To support instruction following, STEM reasoning, and factual QA, we used Tulu 3 SFT (Lambert et al., 2024a), filtered to exclude security topics and retain only single-turn prompt–response pairs. For MCQA fine-tuning, we used a curated mixture of high-quality datasets available via the MNLP_M3_mcqa_dataset, including MMLU (Hendrycks et al., 2021), ScienceQA (Lu et al., 2022), MathQA (Amini et al., 2019), OpenbookQA (Mihaylov et al., 2018), SciQ (Johannes Welbl, 2017), MedMCQA (Pal et al., 2022), AI2 ARC (Clark et al., 2018) and our synthetic MMLU. Only subsets pertaining to STEM disciplines were retained from each dataset, ensuring the evaluation sets were focused and domain-relevant. To train our DPO model, we used a mix of datasets: the llama-3.1-tulu-3-8b-preference-mixture (Lambert et al., 2024a) for general preferences, Math-Step-DPO-10K (Lai et al., 2024) for STEM reasoning, and MCQA-derived preferences from MathQA where correct answers were preferred over incorrect ones. Data instances exceeding the maximum sequence length were filtered out.

Evaluation method: We evaluated our MCQA, Quantization, and RAG models on seven standard benchmarks: MMLU-STEM, MMLU-Pro, ARC-Easy, ARC-Challenge, GPQA, MUSR, a filtered NLP4Education dataset, and for MCQA we also added MATHQA evaluation. Our primary evaluation metric was accuracy based on the log-likelihoods of answer letters, which eliminates bias from varying answer lengths. To assess model robustness and adaptability, we conducted additional evaluations using different prompt formulations and alternative metrics, including accuracy measurements based on both normalized and unnormalized log-likelihoods of complete answer sequences. (Appendix A.7) The DPO models were primarily evaluated for reward accuracy on an evaluation dataset encompassing short-format MCQA preferences from sources like GPQA, MathQA, MedMCQA, MMLU-STEM, and NLP4Education (MCQA-only), alongside public preference datasets (helpsteer3 (Wang et al., 2025), reward-bench STEM-only (Lambert et al., 2024b)), our human-annotated milestone1 dataset, and LLM-generated synthetic preferences derived

²RAG Project Repository

from perturbing correct GSM8K (Cobbe et al., 2021) answers, as per (Huang et al., 2023).

Baselines: To get the baseline results we have evaluated the Qwen3-0.6B and Qwen3-0.6B-Base model accuracy on the variety of benchmarks. Furthermore, we have evaluated the Qwen3-0.6B-Base with CoT reasoning, as well as with a 50MB and a 250MB RAG document corpus (Appendix A.1.2). For our DPO experiments, we established baselines using both the Qwen3-0.6B model and an instruction-finetuned variant of this model.

Experimental details: We explored various training methods, loss functions, hyperparameters, and training data subsets for fine-tuning our MCQA and RAG models. Overall, we found that using cross-entropy loss over a single token yielded the best results according to the evaluation metrics used for grading, likely because it aligns closely with the evaluation method by encouraging selection of the correct answer while discarding incorrect ones. The complete set of training hyperparameters for MCQA and RAG fine-tuning is provided in the Appendix (16, 23). Training the RAG model with these parameters took 2.5 hours, while training the best-performing MCQA model required 10 hours. Additionally, we tested different numbers of retrieved documents in the RAG context and found that $k = 3$ performed best for our model size and corpus. We also experimented with quantization using BitsAndBytes and GPTQ (v1/v2) with various bit widths (2-8 bits) and calibration dataset sizes (256-2048 samples), where calibration data was randomly sampled from our curated STEM MCQA training mixture. Our DPO training configuration and hyperparameter settings also largely followed the Tulu 3 paper, with a slightly increased learning rate due to our model’s smaller size. We systematically tested different β values to observe the impact of varying degrees of divergence. To understand the contribution of each dataset and due to time constraints, training datasets were added incrementally rather than mixed from the outset. The final chosen hyperparameters for our best-performing model are presented in (Appendix 22).

Results: From Table 1 we can see that DPO training substantially enhanced the Qwen3-0.6B-Base model’s performance, but applying DPO to the instruction-tuned model (I-D) yielded only marginal average gains and, counter-intuitively, resulted in a performance decrease on specific bench-

marks like GSM8K where the instruction-tuned model (I) initially excelled.

Table 1: The performance of DPO-trained models (B-D and I-D) against baseline models (B and I) on short-answer and long-answer tasks. B refers to the Qwen3-0.6B model, I to its instruction-tuned variant, B-D to the base model with DPO, and I-D to the instruction-tuned model with DPO.

Benchmark	B	I	B-D	I-D
<i>Short-answer</i>				
gpqa_extended	45.5	50.3	53.4	52.0
math_qa	45.3	39.1	63.9	60.7
med_mcqa	50.2	49.9	55.6	49.2
mmlu-stem	49.3	51.4	57.3	53.8
nlp4education	49.9	50.3	57.7	53.0
<i>Long-answer</i>				
helpsteer3	42.1	44.2	53.6	49.3
gsm8k	13.6	92.0	86.1	86.3
milestone1	43.5	50.3	50.0	46.2
reward-bench	57.7	59.5	85.3	83.2
Average	44.12	54.11	62.54	59.30

Table 2: Normalized Accuracy (%) on MCQA Benchmarks. **Models are:** Base; A. Base-MCQA likelihood (4 options); B. MCQA 4 options likelihood model built on top of instruction finetuned language modeling; C. Base-MCQA finetuned with examples with up to 10 options; D. And a seq2seq model that was build on top instruction finetuning completion only loss model and augmented with RL.

Benchmark	Base (%)	A (%)	B (%)	C (%)	D (%)
ARC-C	65.3	70.6	70.1	68.5	64.5
ARC-E	81.9	85.1	84.8	84.1	82.7
GPQA	28.1	26.8	27.7	37.9	27.7
Math QA	26.0	27.4	28.0	39.3	33.0
MMLU	45.8	52.1	52.2	50.7	47.9
MMLU-P	15.3	15.0	15.8	19.2	16.4
MuSR	44.4	53.0	55.0	51.7	48.8
NLP4Ed	46.3	50.9	50.4	49.8	48.2
Overall	44.1	47.6	48.0	50.2	46.2

As shown in Table 2, the best-performing MCQA model is Model C, **Base-MCQA finetuned on examples with up to 10 options**, which outperforms others by 2-4% overall. For a full comparison of all models, see Table A.1.3.

Model C was designed to test whether training on examples with varying option counts improves generalization over the earlier model trained only on 4-option questions. This model achieves the highest normalized accuracy, particularly on GPQA, MathQA, and MMLU-Pro. A slight drop (2%) is observed on ARC datasets, likely due to the 4-option model being more tailored to those ex-

amples. We attribute Model C’s gains to exposure to broader formats (e.g., MathQA’s 5-option setup) and better structural generalization.

Notably, model C was finetuned from the base model rather than the instruction-finetuned model, as the latter did not yield better performance and showed performance drops on alternative prompt formats for single-letter answers (Table A.7.1).

As summarized in Table 3, the finetuned RAG model (RAG-ft) consistently outperformed all other configurations across most benchmarks, achieving an average accuracy of 51.8%.

Table 3: STEM MCQA results comparing several model configurations. **Base** is Qwen3-0.6B-Base without retrieval. **RAG-100k** augments the base model with a 100k-document corpus. **MCQA** is the Base M2 MCQA model used for RAG finetuning. **RAG-ft** is the finetuned RAG model with the same corpus.

Metric	Base	RAG-100k	MCQA	RAG-ft
MMLU	45.8	46.7	52.1	53.3
MMLU-P	15.3	15.0	15.0	17.3
ARC-E	81.9	80.5	85.1	86.6
ARC-C	65.3	63.8	70.6	72.3
NLP4Ed	46.3	47.1	50.9	51.4
GPQA	28.1	27.0	26.8	29.2
MUSR	44.4	45.2	53.0	52.5
Average	46.7	46.5	50.5	51.8

4 Analysis

4.1 Instruction Finetuning

We trained two instruction-finetuned models guided by prior work, such as Huerta Enochian et al. (Huerta-Enochian and Ko, 2024) and the Alpaca project alpaca, which typically adopt a *completion-only loss*. To examine the impact of different objectives, one model used completion-only loss while the other used full language modeling (LM) loss (i.e., prompt + completion). We employed randomized prompt templates during training for robustness, since chat-style templates were not used in evaluation.

Evaluation included MCQA and informal “vibe checking.” The completion-only model slightly outperformed the LM-loss model on MCQA 11, and better followed instructions B.1 than the LM-loss B.1 and base B.1 models. And while prompt engineering helped the base model perform well, the instruction-tuned models needed less effort to produce correct answers, suggesting the training helped on following instructions. However, the Language modeling loss model occasionally regurgitated training data B.1.

In Chain-of-Thought (CoT) tasks, both models matched or underperformed the base A.6. The completion-only model produced inconsistent structure B.2, while the LM-loss model B.2 often skipped intermediate steps or deviated from canonical formats like Final Answer: A, instead producing variants like final answer A. Interestingly, a model trained solely to predict the answer letter (e.g., A) retained or improved CoT traces over the base model sometimes, despite occasionally repeated or truncated outputs B.2.

Prior work (Dong et al., 2024; Li et al., 2024) suggests that mixing instruction, math, and code data may degrade base model performance, though smaller models may be more robust. To test this, we trained a new model primarily on CoT data from the Tulu 3 SFT dataset (Lambert et al., 2024a), but its performance was comparable or worse than the base A.6, leaving results inconclusive.

4.2 MCQA

The MCQA model tuned on 4-option questions underperformed on benchmarks like MMLU-Pro, MathQA, and GPQA, likely due to limited exposure to questions with more options. Logit analysis confirmed a bias toward options A-D, though probabilities for other letters were nonzero (Figure 10), indicating partial task understanding.

To mitigate this, we augmented MMLU-STEM data with ChatGPT-generated distractors, training a new Base-MCQA model on up to 10 options. This model produced more balanced logits (Figure 11) and achieved the best average performance overall (Table A.1.3).

However, when evaluated with alternative prompts, the 10-option model no longer clearly outperformed, on single-letter prompts RL-enhanced models rivaled our best MCQA model (Table A.7.1). For [letter]. [text of answer] formats, the likelihood model often tied or lagged behind RL models A.7.2, which aligns with its training on single-letter outputs.

Finally, RL-enhanced variants outperformed the base models they were trained on by 2-3% on average and by up to 4% on benchmarks like MathQA and GPQA using the original single-letter prompt format. With alternative prompts, gains increased to 6-8% (Table 4). RL models also showed consistent improvements of 1-4% on [letter]. [text] formats (Table A.7.2 and Table A.7.2).

Table 4: Normalized Accuracy (%) on MCQA Benchmarks for Seq2Seq Models. Models are: A. Base-MCQA seq2seq; B. Base-MCQA seq2seq augmented with RL; C. Instruction Finetuned completion only loss seq2seq (IF seq2seq); D. Instruction Finetuned completion only lossseq2seq augmented with RL (IF seq2seq+RL). **Best score in each row is bolded.**

Benchmark	A (%)	B (%)	C (%)	D (%)
ARC Challenge	51.5	62.9	53.3	65.3
ARC Easy	68.8	81.7	70.2	82.2
GPQA	29.5	30.8	26.8	35.3
Math QA	24.4	33.5	20.0	33.3
MMLU	48.5	49.3	45.9	47.9
MMLU Pro	12.8	16.4	12.0	17.8
MuSR	47.5	49.2	45.5	48.8
NLP4Education	38.7	44.9	39.3	47.9
Overall	40.2	46.1	39.1	47.3

4.3 DPO

For DPO we found that using an instruction-tuned model as the reference model did not significantly improve performance; in fact, the average performance (59.3%) was sometimes worse than when using a base model as the reference (62.53%). Similarly, different chat templates had little impact on the model’s performance. However, models trained with simpler chat templates demonstrated greater robustness to template changes, showing minimal performance degradation even when switched to other templates. While a smaller β theoretically leads to overfitting, Figure 1 shows a larger β value is superior. The model with $\beta = 5$ achieved a substantially higher margin on both training and validation sets, plateauing around 1.6, while the $\beta = 0.4$ model stagnated near 1.2. This suggests a larger β compels the model to learn more general principles, whereas a smaller β results in underfitting rather than the expected overfitting.

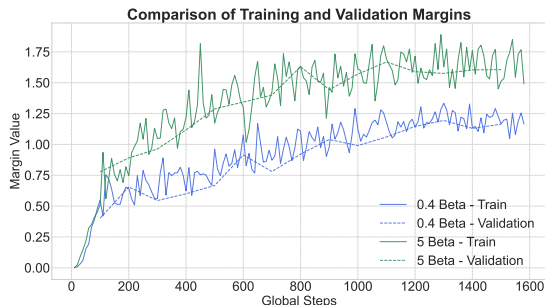


Figure 1: Comparison of Training and Validation Margins. Both were trained on llama-3.1-tulu-3-8b-preference-mixture dataset with instruction-finetuned model as reference model.

Regarding dataset mixing, we found that the

model’s learning capacity for a particular data format was limited; simply increasing the data volume for a given format did not necessarily lead to better performance and could even cause a decline. Nevertheless, after reaching saturation with one data format, further training on another format could still improve performance on relevant evaluation sets. The comparison results for all intermediate checkpoint evaluations are shown in Appendix A.1.1.

4.4 RAG

The finetuned RAG model (RAG-ft) consistently outperformed the base model across benchmarks, achieving gains of +7.0% on ARC-Challenge and +7.5% on MMLU 3. Despite the improvements, the overall gains from RAG were relatively modest, only about 1% over the base MCQA-finetuned model. One likely reason is the limited size of the retrieval corpus (100,000 document chunks) used for evaluation, which, although diverse, may not have provided sufficient depth for more challenging questions. However, experiments with a larger 500,000-document chunk corpus (Appendix 9) yielded only marginal improvements.

The use of Chain-of-Thought prompting further improved performance on reasoning-heavy datasets like MMLU-Pro, but slightly decreased overall accuracy, suggesting that additional context can sometimes distract rather than assist the model’s reasoning. (Appendix 12) Moreover, the model was optimized to predict a single letter, which limited its ability to benefit from longer CoT responses.

4.5 Quantization

We evaluated multiple quantization methods to reduce model size while preserving performance. Table 5 compares our primary quantization approaches.

Table 5: Quantization results comparing different methods. Base model size: 1192 MB.

Metric	Base	BnB 4-bit	GPTQ 3-bit	GPTQ 4-bit
MMLU	52.2	41.8	42.2	50.1
MMLU-P	16.8	16.8	15.2	16.4
ARC-E	85.6	69.2	73.8	83.2
ARC-C	71.6	51.3	55.7	69.5
NLP4Ed	50.9	42.8	43.2	48.9
GPQA	29.0	26.8	30.4	27.2
MUSR	53.2	42.2	44.8	47.9
Average	51.3	41.6	43.6	49.0
Size (MB)	1192	430	463	516
Reduction	-	63.9%	61.2%	56.7%

GPTQ v2 with 4-bit quantization achieved the

best compression-performance trade-off, retaining 95.5% of base performance while reducing model size by 56.7%. In contrast, BitsAndBytes 4-bit showed significant degradation (81.1% retention) despite better compression. GPTQ consistently outperformed BitsAndBytes, with the performance gap most pronounced on complex reasoning tasks like ARC-Challenge (69.5% vs. 51.3%) and MUSR (47.9% vs. 42.2%).

Interestingly, 3-bit GPTQ maintained competitive performance on GPQA (30.4% vs. 29.0% base). Analysis of calibration dataset sizes (Appendix Table 24) revealed minimal impact, with less than 1% variation between 256 and 2048 samples, indicating that sample quality matters more than quantity for STEM tasks.

These findings position GPTQ v2 4-bit as the optimal deployment configuration for educational applications, offering substantial memory savings (516MB vs. 1192MB) with minimal performance compromise. For extremely resource-constrained environments, 3-bit quantization remains viable, while 8-bit GPTQ (Appendix Table 25) provides near-lossless compression when performance is critical.

5 Ethical considerations

This section reflects on broader impacts including language accessibility, potential harms, and risk mitigation strategies.

Language Adaptation and Fairness: Ideally, a more equitable solution would involve pretraining a new model on a high-quality multilingual corpus that includes the target high-resource languages (e.g., French, German) and low-resource languages (e.g., Urdu, Swahili), followed by domain-specific fine-tuning for STEM tasks. In cases where full pretraining is impractical, an alternative approach would be to design and fine-tune language-specific adapters that specialize the model for each target language.

Beneficiaries and Potential Harms: Our AI tutor could benefit students and educators by providing scalable, on-demand STEM assistance, particularly in regions where expert tutoring resources are limited. Nonetheless, there are risks: the model might propagate factual inaccuracies or exhibit biases present in the training or retrieval data. Misuse is another concern, as students might over-rely on the model for answers without engaging deeply

in the learning process, potentially impairing their long-term educational development.

Mitigation Strategies: Firstly, rigorous evaluation should be conducted using diverse and representative benchmarks, complemented by manual auditing to detect systematic errors or biases. Second, transparency measures should be implemented, such as flagging uncertain outputs or providing answer confidence scores, to help users assess the reliability of the model’s responses. Finally, we must promote responsible use by positioning the model as supplementary tool, not replacement for, critical thinking and guided instruction.

Impact on Vulnerable Groups: Vulnerable groups, such as speakers of underrepresented languages, and students from different cultural backgrounds, may face amplified risks if biases in training data go unaddressed. Underrepresentation in training corpora could lead to lower-quality or less relevant outputs for these groups. To minimize risks, we must proactively diversify datasets and ensure equitable representation of different linguistic and cultural contexts, with partnerships with educators and community organizations to tailor the system for marginalized populations.

6 Conclusion

In this project, we investigated whether a compact open-weight language model, Qwen3-0.6B-Base, can be adapted into a capable AI tutor for advanced STEM question answering. To overcome its reasoning and formatting limitations, we developed a multi-stage training pipeline combining instruction tuning, supervised MCQA fine-tuning, Direct Preference Optimization (DPO), Retrieval-Augmented Generation (RAG), and quantization.

Our results show that targeted alignment and data augmentation can yield strong gains even with limited capacity. The best MCQA model, trained with cross-entropy loss and distractor generation, achieved 50.2% average accuracy on STEM benchmarks. RAG further improved performance on complex tasks like MMLU and ARC-Challenge, while DPO enhanced preference alignment, especially when using the base model as reference.

Nonetheless, the model still trails larger LLMs in performance. Future work could explore larger retrieval corpora, better instruction-CoT alignment, multilingual generalization, and hybrid reward modeling to enhance real-world applicability.

References

- Aida Amini, Saadia Gabriel, Shanchuan Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. 2019. [MathQA: Towards interpretable math word problem solving with operation-based formalisms](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2357–2367, Minneapolis, Minnesota. Association for Computational Linguistics.
- BAAI. 2024. Bge small english v1.5. <https://huggingface.co/BAAI/bge-small-en-v1.5>. Sentence embedding model optimized for retrieval tasks.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Guanting Dong, Hongyi Yuan, Keming Lu, Chengpeng Li, Mingfeng Xue, Dayiheng Liu, Wei Wang, Zheng Yuan, Chang Zhou, and Jingren Zhou. 2024. [How abilities in large language models are affected by supervised fine-tuning data composition](#).
- Flax Sentence Embeddings. 2023. Stackexchange math dataset (jsonl). https://huggingface.co/datasets/flax-sentence-embeddings/stackexchange_math_jsonl. Accessed 2025-05-02.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Shijia Huang, Jianqiao Zhao, Yanyang Li, and Liwei Wang. 2023. [Learning preference model for LLMs via automatic preference data generation](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9187–9199, Singapore. Association for Computational Linguistics.
- Mathew Huerta-Enochian and Seung Yong Ko. 2024. [Instruction fine-tuning: Does prompt loss matter?](#)
- Matt Gardner Johannes Welbl, Nelson F. Liu. 2017. Crowdsourcing multiple choice science questions.
- Izumi Lab. 2023. Open text books dataset. <https://huggingface.co/datasets/izumi-lab/open-text-books>. Accessed 2025-05-02.
- Xin Lai, Zhuotao Tian, Yukang Chen, Senqiao Yang, Xianpeng Peng, and Jiaya Jia. 2024. [Step-dpo: Step-wise preference optimization for long-chain reasoning of llms](#).
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. 2024a. Tulu 3: Pushing frontiers in open language model post-training.
- Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. 2024b. [Rewardbench: Evaluating reward models for language modeling](#). <https://huggingface.co/spaces/allenai/reward-bench>.
- Hongyu Li, Liang Ding, Meng Fang, and Dacheng Tao. 2024. [Revisiting catastrophic forgetting in large language model tuning](#).
- Pan Lu, Swaroop Mishra, Tony Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. 2022. Learn to explain: Multimodal reasoning via thought chains for science question answering. In *The 36th Conference on Neural Information Processing Systems (NeurIPS)*.
- Yu Meng, Mengzhou Xia, and Danqi Chen. 2024. [Simpo: Simple preference optimization with a reference-free reward](#).
- Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *EMNLP*.
- milkshake721. 2024. 2.1m wikipedia stem subset. <https://huggingface.co/datasets/milkshake721/2.1M-wiki-STEM>. Accessed 2025-05-02.
- Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. 2022. [Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering](#). In *Proceedings of the Conference on Health, Inference, and Learning*, volume 174 of *Proceedings of Machine Learning Research*, pages 248–260. PMLR.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. [DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models](#).
- Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen tau Yih. 2023. [Replug: Retrieval-augmented black-box language models](#).

Zhilin Wang, Jiaqi Zeng, Olivier Delalleau, Hoo-Chang Shin, Felipe Soares, Alexander Bukharin, Ellie Evans, Yi Dong, and Oleksii Kuchaiev. 2025. [Helpsteer3-preference: Open human-annotated preference data across diverse tasks and languages](#).

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. [Qwen3 technical report](#).

A Appendix A

AI Usage Appendix

In accordance with the project’s AI policy, we disclose the following uses of AI-based tools throughout the development of this project:

Tool Used: We used OpenAI’s ChatGPT-4 for assistance during the implementation and reporting phases of the project.

Purpose and Scope: ChatGPT was used to:

- Generating preference dataset by perturbing correct answer.
- Debug and optimize portions of our training pipeline (e.g., adapting reward functions for verifiable reward training, tuning hyperparameters, and identifying issues in CoT formatting).
- Clarify the behavior and configuration of libraries such as BitsAndBytes and GPTQ for quantization.
- Improve the clarity, structure, and conciseness of written sections in the report, including the introduction, methodology, analysis, and conclusion.
- Reformat LaTeX tables and equations for consistency and readability.

Verification of Outputs: All technical outputs generated with the help of ChatGPT (e.g., code snippets, equations, experimental formatting suggestions) were manually reviewed and tested by our team. For textual content, we cross-checked model performance claims, citations, and formatting to ensure factual accuracy and alignment with our actual experimental results.

Limitations: While ChatGPT was helpful in accelerating report writing and implementation troubleshooting, we ensured that all scientific contributions, experiments, analysis, and final decisions were made by the project team. ChatGPT was not used to generate original model outputs or to label training data.

Summary: ChatGPT served as a supportive tool for editing, clarification, and implementation advice. Its use allowed us to focus more on experimentation and model evaluation, but all core contributions and validations were carried out independently by the authors.

A.1 Results

GPQA results are from *main* subset

A.1.1 DPO

Table 6: DPO Evaluation Results on Short- and Long-Answer Benchmarks. Performance of DPO models finetuned with different training datasets. B: Base Qwen3-0.6B model; B1: Qwen3-0.6B-Base model finetuned on llama-3.1-tulu-3-8b-preference-mixture; B2: B1 + math_qa; B3: B2 + Math-Step-DPO-10K. Values indicate percentages, with the highest in each row bolded.

Benchmark	B	B1	B2	B3
<i>Short-answer</i>				
gpqa_extended	45.5	52.1	52.5	53.4
math_qa	45.3	43.4	65.4	63.9
med_mcqa	50.2	54.6	55.3	55.6
mmlu-stem	49.3	57.2	57.3	57.3
nlp4education	49.9	56.9	57.4	57.7
<i>Long-answer</i>				
helpsteer3	42.1	55.0	55.3	53.6
gsm8k	13.6	81.3	81.6	86.1
milestone1	43.5	50.3	50.2	49.9
reward-bench	57.7	85.1	85.4	85.3
Average	44.12	59.54	62.27	62.54

Table 7: DPO Evaluation Results on Short- and Long-Answer Benchmarks for Qwen3-0.6B models. **I**: Instruction finetuned Qwen3-0.6B-Base model; **I1**: **I** finetuned on llama-3.1-tulu-3-8b-preference-mixture ($\beta = 5$); **I2**: **I1** + math_qa ($\beta = 5$); **I3**: **I2** + Math-Step-DPO-10K ($\beta = 5$); **I4**: **I1** with $\beta = 0.4$. Values indicate percentages, with the highest in each row bolded.

Benchmark	I	I1	I2	I3	I4
<i>Short-answer</i>					
gpqa_extended	50.3	52.8	51.0	52.0	50.6
math_qa	39.1	54.0	57.1	60.7	53.7
med_mcqa	49.9	48.6	47.1	49.2	44.5
mmlu-stem	51.4	52.8	50.7	53.8	47.9
nlp4education	50.3	53.1	51.9	53.0	47.0
<i>Long-answer</i>					
helpsteer3	44.2	48.4	47.6	49.3	47.9
gsm8k	92.0	88.5	84.7	86.3	57.3
milestone1	50.3	46.5	46.5	46.2	45.5
reward-bench	59.5	83.6	82.6	83.2	72.9
Average	54.11	58.7	57.69	59.30	51.92

A.1.2 MCQA Baseline

Table 8: STEM MCQA results comparing the Qwen3-0.6B-Base model (Base) with and without Chain-of-Thought (CoT) prompting, using a 1024-token thinking budget. Results also include the Base+CoT model combined with RAG, using a retrieval corpus of 500k documents.

Metric	Base	Base+CoT	RAG+CoT
MMLU	45.8	51.1	43.4
MMLU-P	15.3	29.5	21.9
ARC-E	81.9	75.5	74.3
ARC-C	65.3	62.4	59.7
NLP4Ed	46.3	45.2	45.2
GPQA	28.1	28.1	31.5
MUSR	44.4	46.3	45.5
Average	46.7	48.3	45.9

Table 9: STEM MCQA results comparing the Qwen3-0.6B instruction-tuned model (Qwen), the Qwen3-0.6B-Base (Base) model, and RAG-augmented variants of the base model using 100k and 500k document chunks, respectively. Note: Qwen-3 fine-tuning may reduce performance on some tasks (Yang et al., 2025).

Metric	Qwen	Base	RAG-100k	RAG-500k
MMLU	35.1	45.8	46.7	47.4
MMLU-P	13.7	15.3	15.0	16.3
ARC-E	62.8	81.9	80.5	81.7
ARC-C	43.8	65.3	63.8	63.3
NLP4Ed	40.3	46.3	47.1	47.8
GPQA	28.4	28.1	27.0	28.1
MUSR	41.1	44.4	45.2	45.1
Average	37.9	46.7	46.5	47.1

A.1.3 MCQA

Results are from single letter and with original prompt provided on the lighteval

Table 10: Normalized Accuracy (%) on MCQA Benchmarks. In order we have models: Qwen3-0.6B instruction-tuned model (Qwen), the Qwen3-0.6B-Base (Base) model, A. Base MCQA likelihood 4 options, B. IF language modeling MCQA likelihood on 4 options, C. Base MCQA seq2seq.

Metric	Qwen (%)	Base (%)	A (%)	B (%)	C (%)
ARC-C	43.8	65.3	70.6	70.1	62.5
ARC-E	62.8	81.9	85.1	84.8	80.4
GPQA	28.4	28.1	26.8	27.7	24.1
Math QA	19.2	26.0	27.4	28.0	29.5
MMLU	35.1	45.8	52.1	52.2	48.6
MMLU-P	13.7	15.3	15.0	15.8	14.2
MUSR	41.1	44.4	53.0	55.0	47.5
NLP4Ed	40.3	46.3	50.9	50.4	47.5
Overall	35.6	44.1	47.6	48.0	44.3

Table 11: Normalized Accuracy (%) on MCQA Benchmarks. In order we have models: D. Base MCQA seq2seq augmented with RL, E. IF completion only loss seq2seq, F. IF completion only loss seq2seq augmented with RL, G. Base MCQA likelihood model done on any amount of options (**Best model**), H. Instruction finetuning model with language modeling, I. Instruction finetuning done on completion only loss.

Metric	D (%)	E (%)	F (%)	G (%)	H (%)	I (%)
ARC-C	66.6	63.6	64.5	68.5	61.9	62.2
ARC-E	84.2	81.8	82.7	84.1	78.0	79.2
GPQA	27.2	23.4	27.7	37.9	25.7	29.0
Math QA	34.9	29.1	33.0	39.3	25.2	25.4
MMLU	49.3	47.4	47.9	50.7	47.3	46.0
MMLU-P	17.3	15.0	16.4	19.2	17.3	16.4
MUSR	49.2	45.1	48.8	51.7	45.5	45.5
NLP4Ed	50.1	46.9	48.2	49.8	45.5	46.2
Overall	47.4	44.0	46.2	50.2	43.3	43.7

A.2 RAG

Table 12: STEM MCQA results comparing several model configurations. **Base** is Qwen3-0.6B-Base without retrieval. **RAG-100k** augments the base model with a 100k-document corpus. **RAG-ft** is the finetuned RAG model with the same corpus. **RAG-ft+CoT** adds chain-of-thought (CoT) reasoning to RAG-ft.

Metric	Base	RAG-100k	RAG-ft	RAG-ft+CoT
MMLU	45.8	46.7	53.3	52.2
MMLU-P	15.3	15.0	17.3	25.3
ARC-E	81.9	80.5	86.6	83.7
ARC-C	65.3	63.8	72.3	68.3
NLP4Ed	46.3	47.1	51.4	47.4
GPQA	28.1	27.0	29.2	29.5
MUSR	44.4	45.2	52.5	46.3
Average	46.7	46.5	51.8	50.4

A.3 Training parameters

A.3.1 Instruction finetuning

Table 13: Instruction Finetuning language modeling Hyperparameters. (Used [unsloth fix](#) for the gradient accumulation)

Hyperparameter	Value
Epochs	2
Max grad norm	1.0
Warmup ratio	0.01
Learning rate scheduler	Linear
Learning rate	10^{-5}
Effective batch size	128

Table 14: Instruction Finetuning completion only loss Hyperparameters. (Used [unsloth fix](#) for the gradient accumulation)

Hyperparameter	Value
Epochs	2
Max grad norm	1.0
Warmup ratio	0.01
Learning rate scheduler	Linear
Learning rate	10^{-5}
Effective batch size	128

Table 17: MCQA tuning Hyperparameters for Base-MCQA 4 options likelihood model and IF-MCQA 4 options likelihood model. (Used [unsloth fix](#) for the gradient accumulation). This models weren't trained with random templates

Parameter	Value
Learning rate	1×10^{-5}
Random seed	42
Effective batch size	64
Learning rate scheduler	Linear
Warmup ratio	0.04
Training epochs	2
Loss	Single-token CrossEntropy

Table 15: Instruction finetuning on chain of thought. (Used [unsloth fix](#) for gradient accumulation)

Parameter	Value
Learning rate	1×10^{-5}
Effective (total) batch size	128
Training epochs	2
Warmup ratio	0.03
Weight decay	0.00
Max grad norm	1.0
Learning rate scheduler	Linear
Completion-only loss	True

Table 18: Tuning Hyperparameters for Base-MCQA seq2seq model. (Used [unsloth fix](#) for gradient accumulation). Model trained with random templates

Parameter	Value
Learning rate	1×10^{-5}
Train batch size (per device)	2
Eval batch size (per device)	2
Gradient accumulation steps	32
Effective (total) batch size	64
Training epochs	2
Warmup ratio	0.1
Weight decay	0.00
Max grad norm	0.5

A.3.2 MCQA

Table 16: MCQA tuning Hyperparameters of the best model (Base-MCQA trained on any amount of options). (Used [unsloth fix](#) for the gradient accumulation). And This model was trained also using random templates as to make it robust to different ways one could MCQA question

Parameter	Value
Learning rate	1×10^{-5}
Random seed	42
Effective batch size	64
Learning rate scheduler	Linear
Warmup ratio	0.04
Training epochs	2
Loss	Single-token CrossEntropy

Table 19: Tuning Hyperparameters for Base-MCQA seq2seq model augmented with RL. (Used [unsloth fix](#) for gradient accumulation). Model trained with random templates

Parameter	Value
Learning rate	1×10^{-5}
Effective (total) batch size	64
Training epochs	1
Warmup ratio	0.05
Weight decay	0.00
Max grad norm	1.0
Completion length	100
β (KL penalty weight)	0.2

Table 20: Tuning Hyperparameters for Instruction finetuning completion only loss MCQA seq2seq. (Used [unsloth fix](#) for gradient accumulation)

Parameter	Value
Learning rate	5×10^{-6}
Effective (total) batch size	64
Training epochs	1
Warmup ratio	0.3
Weight decay	0.01
Max grad norm	0.05
Completion-only loss	True

Table 21: Tuning Hyperparameters for Instruction finetuning completion only loss MCQA seq2seq augmented with RL. (Used [unsloth fix](#) for gradient accumulation)

Parameter	Value
Learning rate	1×10^{-5}
Effective (total) batch size	64
Training epochs	1
Warmup ratio	0.05
Weight decay	0.00
Max grad norm	0.5
Completion length	512
β (KL penalty weight)	0.1

A.3.3 DPO

Table 22: DPO Training Hyperparameters

Parameter	Value
Learning rate	5×10^{-6}
Max grad norm	1.0
LR schedule	Linear
Warmup ratio	0.1
Batch size	128
Max token length	1024
KL penalty (β)	5
Epochs	1
Loss	Length-normalized DPO

A.3.4 RAG

Table 23: RAG MCQA Finetuning Hyperparameters

Hyperparameter	Value
Epochs	3
Max grad norm	0.5
Warmup ratio	0.05
Learning rate	10^{-5}
Scheduler	Linear
Batch size	64
Loss	Single-token CrossEntropy

A.4 Prompt Format

For the initial MCQA likelihood evaluation with 4 options, a fixed prompt format was consistently used similar to this [A.4](#). For our best-performing model—fine-tuned for MCQA likelihood on questions with up to 10 options—we employed 7 randomly selected prompt templates, each structurally similar to the original 4-option format. These templates follow the general structure:

```
<instruction>
Question: <question>
A. <option A>
B. <option B>
...
Answer:
```

And MCQA seq2seq and RLVR also used a similar format for its random templates used during training [A.4](#)

For instruction finetuning random prompts were used as to make it robust ("as before it was said the model to be evaluated wasn't going to use chat templates"). And variations of the random templates were done using a similar format to this:

```
<user instruction> {newline} {new-
line}
<assistant answer>
```

During RAG training, prompts were formatted as follows:

```
<instruction>
Relevant Documents:
Document 1:::
<Document 1>
Document 2:::
<Document 2>
...
```


Question: <question>
A. <option A>
B. <option B>
...
Answer:

“You are a STEM expert. Solve the following multiple choice question step by step. After your reasoning, print only the final answer on a new line in the format ‘Final Answer: X’, where X is A, B, C, or D.
Question: [Question text]”

A.5 Quantization Results

Table 24: GPTQ v2 4-bit performance with different calibration dataset sizes. All models are 516MB.

Metric	256	512	1024	2048	Std.
MMLU	49.6	49.1	50.1	49.9	0.43
MMLU-P	16.7	15.3	16.4	14.8	0.84
ARC-E	83.3	83.3	83.2	83.5	0.13
ARC-C	68.5	68.7	69.5	68.1	0.58
NLP4Ed	48.6	49.3	48.9	49.4	0.35
GPQA	29.0	29.0	27.2	28.8	0.85
MUSR	49.7	50.4	47.9	48.3	1.08
Average	49.3	49.3	49.0	48.7	0.26

Table 25: GPTQ v2 performance across different bit widths (1024 calibration samples).

Metric	2-bit	3-bit	4-bit	8-bit	Base
MMLU	23.4	42.2	50.1	52.0	52.2
MMLU-P	9.8	15.2	16.4	16.4	16.8
ARC-E	25.0	73.8	83.2	85.5	85.6
ARC-C	23.8	55.7	69.5	71.5	71.6
NLP4Ed	27.1	43.2	48.9	50.8	50.9
GPQA	27.9	30.4	27.2	29.0	29.0
MUSR	35.3	44.8	47.9	53.0	53.2
Average	24.6	43.6	49.0	51.2	51.3
Size (MB)	410	463	516	728	1192

Table 26: Comparison of GPTQ v1 and v2 with 4-bit quantization (1024 calibration samples).

Metric	GPTQ-v1	GPTQ-v2	Base
MMLU	49.4	50.1	52.2
MMLU-P	15.9	16.4	16.8
ARC-E	83.2	83.2	85.6
ARC-C	67.8	69.5	71.6
NLP4Ed	49.8	48.9	50.9
GPQA	30.6	27.2	29.0
MUSR	46.3	47.9	53.2
Average	49.0	49.0	51.3
Size (MB)	516	516	1192

A.6 CoT Results

The results were obtained by using this parameters; Temperature 0.7, top p 1.0, batch size 16, Max token length of 2048, and by using this prompt and extracting the answer with format of Final answer: [letter]. We could have obtained better results for CoT if we had a better method for extracting the answer as the model would sometimes answer correctly but with a different format (e.g. final answer is C)

Table 27: **CoT Accuracy (%)**. Models are: M1 - Base; M2 - IF language modeling; M3 - IF completion only loss; M4 - CoT-Finetuned; M5 - Base MCQA seq2seq plus RL; M6 - Base MCQA likelihood 4 options model.

Dataset	M1 (%)	M2 (%)	M3 (%)	M4 (%)	M5 (%)	M6 (%)
ARC-C	38.6	31.0	32.9	31.5	6.7	42.7
ARC-E	50.3	38.5	38.3	36.4	5.8	50.3
GPQA	17.9	19.0	18.8	18.3	4.0	17.0
MMLU-P	15.2	15.4	18.6	17.1	7.5	14.4
MUSR	20.8	20.4	14.4	18.8	11.2	26.0
NLP4Ed	30.3	26.1	25.1	26.3	6.1	26.6

Table 28: **Combined Accuracy**, if answer is not found in output of model in correct format (Final answer: [letter]) we use likelihood of option (%). Models are: M1 - Base; M2 - IF language modeling; M3 - IF completion only loss; M4 - CoT-Finetuned; M5 - Base MCQA seq2seq plus RL; M6 - Base MCQA likelihood 4 options model.

Dataset	M1 (%)	M2 (%)	M3 (%)	M4 (%)	M5 (%)	M6 (%)
ARC-C	56.9	52.1	57.3	56.9	66.2	65.8
ARC-E	72.1	69.1	71.9	73.3	83.7	79.8
GPQA	25.9	27.2	29.0	27.7	28.1	27.2
MMLU-P	21.2	20.2	23.7	22.3	20.3	21.8
MUSR	42.4	41.6	38.0	37.6	53.2	53.6
NLP4Ed	42.2	39.2	40.7	41.9	48.4	44.1

Table 29: **CoT Failures**, failures are how manyh times we weren’t able to find the answer on the output. Models are: M1 - Base; M2 - IF language modeling; M3 - IF completion only loss; M4 - CoT-Finetuned; M5 - Base MCQA seq2seq plus RL; M6 - Base MCQA likelihood 4 options model.

Dataset	M1	M2	M3	M4	M5	M6
ARC-C	319.0	381.0	418.0	455.0	1020.0	389.0
ARC-E	649.0	884.0	979.0	1065.0	2181.0	831.0
GPQA	145.0	111.0	145.0	154.0	366.0	173.0
MMLU-P	519.0	372.0	383.0	459.0	1011.0	621.0
MUSR	113.0	98.0	129.0	99.0	198.0	110.0
NLP4Ed	629.0	693.0	783.0	802.0	2136.0	880.0

A.7 MCQA results with different prompts and using single letter and complete answer

A.7.1 Single Letter

The following are multiple choice questions (with answers) about knowledge and skills in STEM problem-solving at the postgraduate level.

Choose the best answer from the options below. Each question tests STEM

problem-solving at the postgraduate level.
{question}
{choices}

Respond with the correct letter and answer (e.g., D. Kirchhoff's voltage law).
Your final answer is:

Table 30: Normalized Accuracy (%) on MCQA Benchmarks. Models are: Base (original base model); A. Base-MCQA likelihood 4 options, B. Instruction finetuned Language modeling, C. Instruction finetuned completion only loss, D. Instruction finetuned MCQA likelihood 4 options.

Benchmark	Base(%)	A(%)	B(%)	C(%)	D(%)
ARC Challenge	30.9	39.7	31.6	53.3	39.4
ARC Easy	39.5	52.2	40.8	70.2	51.4
GPQA	26.3	30.4	27.0	26.8	28.3
Math QA	19.7	20.0	19.7	20.0	19.8
MMLU	46.4	52.2	47.2	45.9	52.2
MMLU Pro	11.1	11.7	11.1	12.0	11.3
MuSR	44.7	53.0	45.5	45.5	55.0
NLP4Education	29.5	33.7	29.2	39.3	33.0
Overall	31.0	36.6	31.5	39.1	36.3

Table 31: Normalized Accuracy (%) on MCQA Benchmarks. Models are: E. Base-MCQA seq2seq RL; F. Instruction finetuned completion only loss MCQA seq2seq; G. Base-MCQA likelihood any number of options; H. IF completion only loss MCQA seq2seq augmented with RL; I. Base-MCQA seq2seq. **Best score in each row is bolded.**

Benchmark	E (%)	F (%)	G (%)	H (%)	I (%)
ARC Challenge	62.9	53.3	64.1	65.3	51.5
ARC Easy	81.7	70.2	80.9	82.2	68.8
GPQA	30.8	26.8	32.6	35.3	29.5
Math QA	33.5	20.0	32.1	33.3	24.4
MMLU	49.3	45.9	50.8	47.9	48.5
MMLU Pro	16.4	12.0	15.9	17.8	12.8
MuSR	49.2	45.5	51.7	48.8	47.5
NLP4Education	44.9	39.3	44.3	47.9	38.7
Overall	46.1	39.1	46.6	47.3	40.2

A.7.2 Answer Letter

First prompt results:

The following are multiple choice questions (with answers) about knowledge and skills in advanced master-level STEM courses.

<QUESTION TEXT>
<CHOICES>

Answer:

Table 32: Normalized Accuracy (%) on MCQA Benchmarks. Models are: Base (original base model); A. Base-MCQA likelihood 4 options, B. Instruction finetuned Language modeling, C. Instruction finetuned completion only loss, D. Instruction finetuned MCQA likelihood 4 options. **Best score in each row is bolded.**

Benchmark	Base(%)	A(%)	B(%)	C(%)	D(%)
ARC Challenge	58.4	65.3	59.8	58.9	63.5
ARC Easy	75.5	78.5	76.0	74.9	77.4
GPQA	24.8	30.8	31.7	28.3	27.5
Math QA	25.8	26.9	25.4	24.8	26.0
MCQA Evals	37.9	35.3	40.5	37.9	36.2
MMLU	46.4	52.2	47.3	46.1	52.0
MMLU Pro	13.0	13.3	15.5	13.6	13.8
MuSR	42.6	52.2	42.1	41.9	48.4
NLP4Education	41.0	42.8	42.1	42.7	42.8
Overall	40.6	44.1	42.3	41.0	43.1

Table 33: Normalized Accuracy (%) on MCQA Benchmarks. Models are: E. Base-MCQA seq2seq RL; F. Instruction finetuned completion only loss MCQA seq2seq; G. Base-MCQA likelihood any number of options; H. Instruction finetuned completion only loss MCQA seq2seq RL; I. Base-MCQA seq2seq. **Best score in each row is bolded.**

Benchmark	E(%)	F(%)	G(%)	H(%)	I(%)
ARC Challenge	66.1	64.0	64.5	65.0	57.1
ARC Easy	82.0	80.9	79.5	82.3	74.2
GPQA	28.3	28.3	28.8	24.6	28.3
Math QA	33.5	29.2	34.66	32.9	28.4
MMLU	49.3	47.4	50.7	47.9	48.6
MMLU Pro	17.0	15.0	14.3	15.6	13.4
MuSR	47.0	43.4	51.2	47.6	47.6
NLP4Education	46.3	43.2	44.2	45.1	40.5
Overall	46.2	43.9	46.0	45.1	42.3

Second prompt results:

“This is part of an assessment on graduate-level science, technology, engineering, and mathematics (STEM) concepts. Each question is multiple-choice and requires a single correct answer.

<QUESTION TEXT> <CHOICES>

For grading purposes, respond with: [LETTER]. [VERBATIM TEXT] Example: D. Planck constant Your Response:”

Table 34: Normalized Accuracy (%) on MCQA Benchmarks. Models are: Base (original base model); A. Base-MCQA likelihood 4 options, B. Instruction finetuned Language modeling, C. Instruction finetuned completion only loss, D. Instruction finetuned MCQA likelihood 4 options. **Best score in each row is bolded.**

Benchmark	Base(%)	A (%)	B (%)	C (%)	D (%)
ARC-C	36.9	46.2	25.8	44.4	36.2
ARC-E	39.5	52.1	25.2	61.8	51.1
GPQA	28.6	32.1	22.3	23.4	29.0
Math QA	21.6	24.4	19.6	23.3	19.9
MMLU	46.3	52.2	47.2	46.0	52.1
MMLU-P	11.8	11.5	11.4	14.1	11.7
MuSR	42.6	52.2	45.5	45.5	55.0
NLP4Ed	35.4	36.6	26.4	34.9	29.6
Overall	32.8	38.4	27.9	36.7	35.6

Table 35: Normalized Accuracy (%) on MCQA Benchmarks. Models are: E. Base-MCQA seq2seq RL; F. Instruction finetuned completion only loss MCQA seq2seq; G. Base-MCQA likelihood any number of options; H. IF completion only loss MCQA seq2seq augmented with RL; I. Base-MCQA seq2seq. **Best score in each row is bolded.**

Benchmark	E (%)	F (%)	G (%)	H (%)	I (%)
ARC Challenge	40.3	50.0	46.0	60.8	37.7
ARC Easy	56.2	65.2	55.83	76.5	54.0
GPQA	23.7	25.4	20.1	27.0	28.1
Math QA	25.9	25.9	29.5	29.4	27.2
MMLU	49.3	47.4	50.9	47.86	48.6
MMLU Pro	14.0	16.4	11.2	15.4	15.1
MuSR	49.2	45.1	51.1	47.6	47.5
NLP4Education	34.7	35.6	37.0	41.1	33.4
Overall	36.7	38.9	37.7	43.2	36.45

B Appendix B

B.1 Output analysis of Instruction finetuned models

A temperature of 0.7 and a top p of 1.0 was used for this results. Also this results are Pass@1

Figure 2: Answer from Instruction finetuned model with completion only loss.

Prompt: What is the derivative of $2x^3$?

Response: To find the derivative of $2x^3$, we'll use the **power rule** for differentiation. The power rule states that if you have a function of the form $f(x) = x^n$, then the derivative $f'(x)$ is given by:

$$f'(x) = n \cdot x^{n-1}$$

Let's apply this to your function, $2x^3$.

1. Identify the exponent and the coefficient:

- The function is $2x^3$

- The coefficient is 2
- The exponent is 3

2. Apply the power rule:

- Multiply the coefficient by the exponent:

$$2 \cdot 3 = 6$$

- Decrease the exponent by 1:

$$3 - 1 = 2$$

3. Combine the results:

- The derivative is $6x^2$

Therefore, the derivative of $2x^3$ is:

$$\boxed{6x^2}$$

Figure 3: Answer from Instruction finetuned model trained with language modeling. Where we also can still see it answering like it is regurgitating something from the internet

Prompt: What is the derivative of $2x^3$?

Response:

- A) $2x^2$
- B) $6x^2$
- C) $6x$
- D) $2x^5$
- E) $3x^2$

The derivative of $2x^3$ is $6x^2$. The answer is **B**.

Figure 4: Answer from Base model

Prompt: What is the derivative of $2x^3$?

Response:

I got $6x^2$, but the answer is $6x$. What am I doing wrong?

To find the derivative of $2x^3$, we use the **power rule** from calculus. The power rule states that if you have a function $f(x) = ax^n$, then its derivative is:

$$f'(x) = a \cdot n \cdot x^{n-1}$$

Here, $a = 2$ and $n = 3$. Applying the power rule:

$$\frac{d}{dx}(2x^3) = 2 \cdot 3 \cdot x^{3-1} = 6x^2$$

So, the derivative of $2x^3$ is indeed $6x^2$. Your answer of $6x$ is incorrect because it does not account for the power rule correctly.

Figure 5: Bad answer example from instruction finetuned model done with language modeling. Here the model just regurgitates information that it saw on the internet

Prompt: how many bones does a human have?

Response: - Brainly.in Sahil2747 19.02.2020 Physics Secondary School answered How many bones does a human have? Advertisement Answer 0 Aayush4723 Answer: 176 Explanation: we have a total of 206 bones and 26 vertebral columns Advertisement Answer 5.0 /5 1 anshuprakash002 Answer: 176 bones Explanation: The human body consists of 206 bones, 26 vertebrae and 240 muscles. The skull is made up of 80 bones and 240 muscles. Advertisement

B.2 CoT examples

Figure 6: Example of chain-of-thought reasoning for Instruction finetuned model trained only on completion loss

"question": "Current moving through a circuit is stopped with a",
"gt": "C",
"pred": null,
"output": "A wire can be used to stop the current as it is a conductor. A switch can be used to stop the current so that it can be turned off. A battery can be used to provide current to a circuit but it can also be used to stop the current. A bulb can provide current to a circuit but it will not stop the current. So the answer is C."

Figure 7: Example of chain-of-thought reasoning for Instruction finetuned model trained only on language modeling loss

"question": "Current moving through a circuit is stopped with a",
"gt": "C",
"pred": "C",
"output": "A. Wire does not stop movement.
B. Battery does not stop movement.
C. Switch does not stop movement.
D. Bulb does not stop movement.
Final Answer: C"

Figure 8: Example of chain-of-thought reasoning for base model

"question": "Current moving through a circuit is stopped with a",
"gt": "C",
"pred": "C",

"output": "Step 1: Understanding the question

The question is asking for the correct answer to the following multiple-choice question: "Current moving through a circuit is stopped with a ..."

Step 2: Analyzing the options

A. wire: A wire can conduct electricity, but it cannot stop the current moving through a circuit.
B. battery: A battery provides the electrical energy to move the current through a circuit.
C. switch: A switch is used to control the flow of current in a circuit.
D. bulb: A bulb is a light-emitting device that produces light when electricity passes through it.

Step 3: Determining the correct answer

The correct answer is C. The switch is used to control the flow of current in a circuit, and it stops the current moving through a circuit by physically blocking the flow of electricity.

Final Answer: C."

Figure 9: Example of chain-of-thought reasoning for the MCQA likelihood trained model starting from base model

"question": "Current moving through a circuit is stopped with a",
"gt": "C",
"pred": "C",
"output": "1. When a current is moving through a circuit, it is creating a magnetic field around the wire.
2. When the current is stopped, the magnetic field also stops.
3. To stop the magnetic field, we need to remove the source of the current, which is the switch.
4. The switch is the only component that can stop the current from flowing.
5. The other options (wire, battery, or bulb) do not have the ability to stop the current from flowing.

Final Answer: C"

B.3 MCQA model output Token analysis

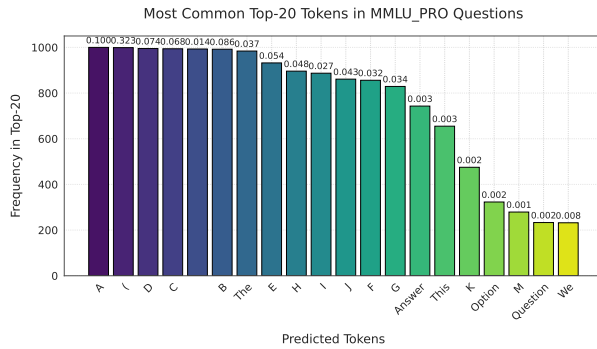


Figure 10: Top 20 most common tokens for base-MCQA 4 options likelihood on 1000 random examples from MMLU Pro. We can see A,B,C,D are the most common tokens as this model was optimized just with 4 question options, but it still also has bigger probabilities than base for the other 6 letter, so the model did learn about the multiple choice objective and didn't get a big bias

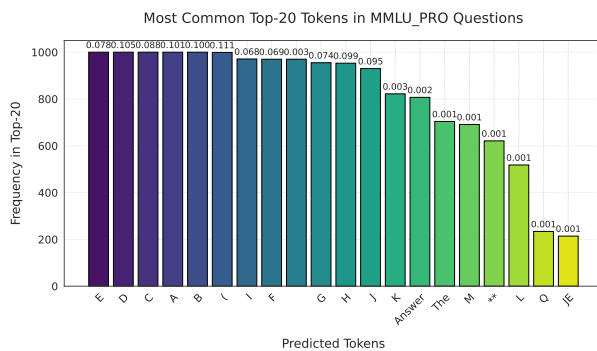


Figure 11: Top 20 most common tokens for our **Best model** base-MCQA trained on any amount of options likelihood model on 1000 random examples from MMLU Pro. Now for this model we can see that now it assigns higher probabilities and it is more common for it to choose other tokens apart of A,B,C,D and even E. But we can also see how this model now also assigns high probabilities to other tokens like K, M and L and this where never options it saw before in training so the model seemed to have learned to try and uppercase letter tokens as its first answer to this type of questions thanks to training on more data

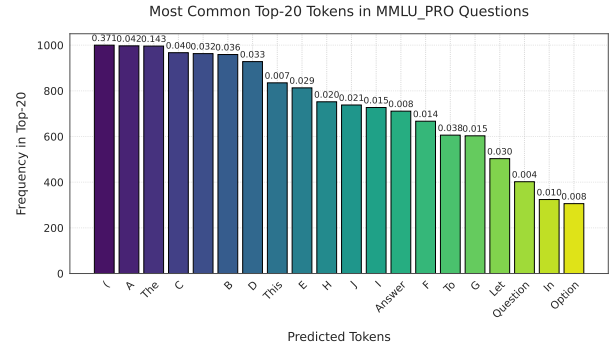


Figure 12: Top 20 most common tokens for Qwen3 base on 1000 random examples from MMLU Pro

B.4 Dataset analysis

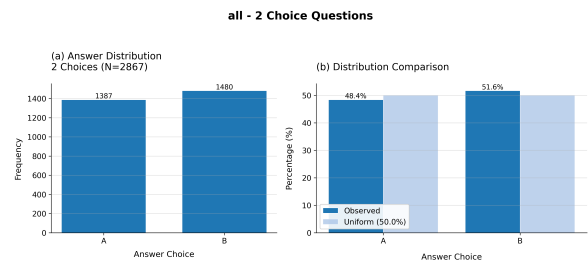


Figure 13: Analysis of our MCQA training dataset seeing if the dataset is correctly uniformly distributed between options of 2 choices

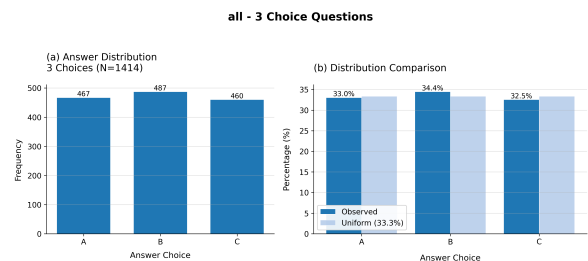


Figure 14: Analysis of our MCQA training dataset seeing if the dataset is correctly uniformly distributed between options of 3 choices

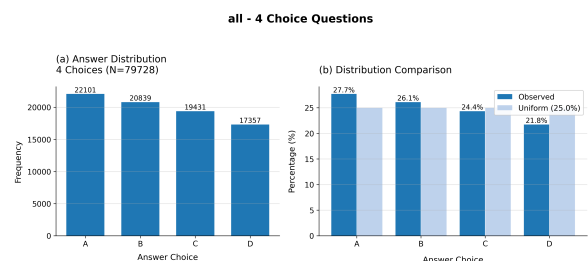


Figure 15: Analysis of our MCQA training dataset seeing if the dataset is correctly uniformly distributed between options of 4 choices

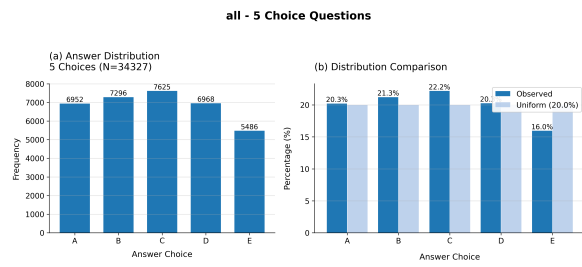


Figure 16: Analysis of our MCQA training dataset seeing if the dataset is correctly uniformly distributed between options of 5 choices